

Universidad de las Fuerzas Armadas ESPE
Matriz – Sangolquí



INGENIERÍA DE SOFTWARE
ANÁLISIS Y DISEÑO DE SOFTWARE

TEMA: Informe Plan de Pruebas

Docente: Ing. Jenny Ruiz

NRC: 30724

Integrantes:

- Blacio Julio
- Solano Josue
- Sigcha Kenned
- Toapanta Alexander

Sangolquí, julio 2026

índice

Sistema IronClad Box - Gestión Operativa de Gimnasio CrossFit	3
RESUMEN EJECUTIVO	3
Estado de Implementación	3
Requisitos Funcionales - Estado Final	3
1. OBJETIVO DE LAS PRUEBAS	5
1.1 Objetivo General	5
1.2 Objetivos Específicos	5
2. PLANTEAMIENTO	6
2.1 Alcance de las Pruebas	6
2.2 Enfoque de Testing	7
2.3 Estrategia de Mocking	7
3. HERRAMIENTA	8
3.1 Framework de Testing	8
3.2 Entorno de Pruebas	8
3.3 Estructura de Archivos	8
4. IMPLEMENTACIÓN POR REQUISITO	9
4.1 REQ-001: Creación de Cuentas	9
1. Lógica de Creación en UsuarioService:	10
2. Tests de Unidad en PHPUnit:	10
4.2 REQ-002: Edición de Cuentas	11
1. Lógica de Edición en UsuarioService:	11
2. Tests de Unidad en PHPUnit:	12
4.3 REQ-003: Desactivación de Cuentas	13
1. Lógica en UsuarioService:	13
2. Tests de Unidad en PHPUnit:	13
4.4 REQ-004: Inicio de Sesión	14
1. Método de Autenticación en UsuarioService:	14
2. Tests de Unidad en PHPUnit:	15
4.5 REQ-005: Validación de Estado de Usuario	16
1. Validación de Estado en Autenticación:	16
2. Restricción en Builder:	16
4.6 REQ-006: Navegación Según Rol	17
1. Control de Autenticación y Cierre de Sesión en Controlador:	17
2. Tests de Unidad en PHPUnit:	17
4.7 REQ-007: Creación de Membresías	18
1. Lógica de Creación en MembresiaService:	18
4.8 REQ-008: Edición de Membresías	19
1. Reconstrucción y Mutación en Builder:	19
2. Tests de Unidad en PHPUnit:	19
4.9 REQ-009: Asignación de Membresías	20
1. Método Actualizar Asignación en Servicio:	20
4.10 REQ-010: Registro de Pagos	21

1. Lógica de Servicio en MembresiaService:	21
2. Tests de Unidad en PHPUnit:	22
4.11 REQ-011: Consulta de Membresías	23
1. Consulta y Cancelación en MembresiaService:	23
2. Tests de Unidad en PHPUnit:	23
4.12 REQ-012: Control de Vencimientos	24
1. Cálculo de Vencimiento y Anti-duplicados:	24
2. Tests de Unidad en PHPUnit:	25
4.13 REQ-013: Creación de Clases	26
1. Construcción Validada de Clases en ClaseBuilder:	26
2. Tests de Unidad en PHPUnit:	27
4.14 REQ-014: Edición de Clases	27
1. Método de Edición en ClaseService:	28
4.15 REQ-015: Eliminación de Clases	28
1. Eliminación Controlada en ClaseService:	28
2. Tests de Unidad en PHPUnit:	29
4.16 REQ-016: Asignación de Entrenadores	29
1. Validación de Disponibilidad de Entrenador:	30
2. Tests de Unidad en PHPUnit:	30
4.17 REQ-017: Control de Cupos	30
1. Lógica de Reserva y Verificación de Membresía:	30
2. Tests de Unidad en PHPUnit:	31
4.18 REQ-018: Validación de Horarios	33
1. Detección de Solapamientos e Ignorar ID Propio:	33
2. Tests de Unidad en PHPUnit:	34
5. RESULTADOS OBTENIDOS	35
5.1 Resumen de Ejecución	35
5.2 Desglose por Archivo	35
5.3 Métricas por Requisito	36
5.4 Gráfico de distribucion de Tests	37
6. PROBLEMAS Y SOLUCIONES	38
6.1 Ejecución de PHPUnit con sesiones PHP	38
6.2 Validación de contraseñas hasheadas	39
6.3 Validación de cédula ecuatoriana	40
6.4 Pruebas sin dependencia directa de MySQL/Aiven	41
6.5 Fechas futuras en la creación de clases	42
6.6 Validación de horarios solapados	43
6.7 Control de cupos y reservas duplicadas	45
7. CONCLUSIONES	45
7.1 Logros Alcanzados	45
7.2 Calidad del Software	46
7.3 Lecciones Aprendidas	46
7.4 Recomendaciones Futuras	47
7.5 Resumen Final	48

ANEXOS	49
Anexo A: Comandos de Ejecución	49
Anexo B: Estructura de Test Típica	50
Anexo C: Configuración Completa	51
Anexo D: Métricas Detalladas	52
Anexo E: Evidencia de Ejecución	53
FIN DEL INFORME	56

Sistema IronClad Box - Gestión Operativa de Gimnasio CrossFit

Proyecto: IronClad Box - Sistema de Gestión Operativa de Gimnasio CrossFit

Tipo de Documento: Plan de Pruebas Unitarias y Resultados

Fecha de Creación: Julio 2026

Fecha de Ejecución: 8 de julio de 2026

Versión: 1.0

Estado: COMPLETADO (100%)

RESUMEN EJECUTIVO

Estado de Implementación

Métrica	Valor
Tests Ejecutados	72/72 (100% pasando)
Aserciones Ejecutadas	128
Tiempo de Ejecución	1.353s
Suites de Tests	9/9 (100% pasando)

Requisitos Implementados	18/18 (100%)
Tasa de Éxito	100%

Requisitos Funcionales - Estado Final

RF	Nombre	Estado	Tests	Tasa Éxito
REQ-001	Creación de Cuentas	COMPLETO	7	100%
REQ-002	Edición de Cuentas	COMPLETO	7	100%
REQ-003	Desactivación de Cuentas	COMPLETO	2	100%
REQ-004	Inicio de Sesión	COMPLETO	6	100%
REQ-005	Validación de Estado de Usuario	COMPLETO	2	100%
REQ-006	Navegación Según Rol	COMPLETO	3	100%
REQ-007	Creación de Membresías	COMPLETO	4	100%
REQ-008	Edición de Membresías	COMPLETO	4	100%
REQ-009	Asignación de Membresías	COMPLETO	4	100%
REQ-010	Registro de Pagos	COMPLETO	3	100%
REQ-011	Consulta de Membresías	COMPLETO	2	100%
REQ-012	Control de Vencimientos	COMPLETO	4	100%
REQ-013	Creación de Clases	COMPLETO	6	100%
REQ-014	Edición de Clases	COMPLETO	4	100%
REQ-015	Eliminación de Clases	COMPLETO	3	100%
REQ-016	Asignación de Entrenadores	COMPLETO	3	100%
REQ-017	Control de Cupos	COMPLETO	5	100%
REQ-018	Validación de Horarios	COMPLETO	3	100%
TOTAL	18 Requisitos Funcionales	COMPLETO	72	100%

1. OBJETIVO DE LAS PRUEBAS

1.1 Objetivo General

Validar que los 18 requisitos funcionales definidos en el tablero Kanban del sistema IronClad Box cumplan con las especificaciones establecidas, garantizando la correcta funcionalidad de:

- Gestión de usuarios: creación, edición, desactivación, inicio de sesión, validación de estado y navegación según rol.
- Gestión de membresías: creación, edición, asignación a atletas, registro de pagos, consulta y control de vencimientos.
- Gestión de clases: creación, edición, eliminación lógica, asignación de entrenadores, control de cupos y validación de horarios.
- Patrón Builder para la construcción validada de entidades principales antes de llegar a la capa DAO.
- Reglas de negocio implementadas en la capa de servicios mediante arquitectura MVC por capas.

1.2 Objetivos Específicos

1. Validar Builders: Verificar que UsuarioBuilder, MembresiaBuilder y ClaseBuilder construyan entidades válidas y rechacen datos incompletos, inválidos o inconsistentes.
2. Validar servicios: Asegurar que UsuarioService, MembresiaService y ClaseService manejen correctamente la lógica de negocio, validaciones, errores y coordinación con la capa DAO.
3. Validar usuarios y autenticación: Comprobar la creación, edición, desactivación, inicio de sesión, validación de usuarios activos/inactivos y navegación según rol.
4. Validar membresías y pagos: Confirmar la creación, edición, asignación, registro de pagos, consulta de estado y cálculo de vencimientos de membresías.
5. Validar clases y reservas: Verificar la creación, edición, eliminación lógica, asignación de entrenadores, control de cupos y validación de horarios solapados.
6. Validar el patrón Builder: Asegurar que las entidades Usuario, Membresia y Clase no puedan construirse con datos inválidos antes de ser procesadas por servicios o persistidas en MySQL.

2. PLANTEAMIENTO

2.1 Alcance de las Pruebas

El plan de pruebas cubre los 18 requisitos funcionales registrados en el tablero Kanban de IronClad Box, agrupados en tres módulos principales: usuarios, membresías y clases.

- Módulo de usuarios:
 - REQ-001 Creación de cuentas
 - REQ-002 Edición de cuentas
 - REQ-003 Desactivación de cuentas
 - REQ-004 Inicio de sesión
 - REQ-005 Validación de estado de usuario
 - REQ-006 Navegación según rol
- Módulo de membresías:
 - REQ-007 Creación de membresías
 - REQ-008 Edición de membresías
 - REQ-009 Asignación de membresías
 - REQ-010 Registro de pagos
 - REQ-011 Consulta de membresías
 - REQ-012 Control de vencimientos
- Módulo de clases:
 - REQ-013 Creación de clases
 - REQ-014 Edición de clases
 - REQ-015 Eliminación de clases
 - REQ-016 Asignación de entrenadores
 - REQ-017 Control de cupos
 - REQ-018 Validación de horarios

Las pruebas unitarias cubren:

- Builders: validación de datos, construcción de entidades, manejo de campos obligatorios y reglas del patrón Builder.
- Servicios: reglas de negocio, validaciones cruzadas, manejo de excepciones y coordinación con DAO.
- Controladores de autenticación: validación de sesión, login, logout y usuario autenticado.
- Reglas funcionales: usuarios activos/inactivos, roles, membresías vigentes, pagos, cupos disponibles, reservas duplicadas y horarios solapados.

2.2 Enfoque de Testing

Metodología: Test-Driven Development (TDD) adaptado.

Tipo de Pruebas: Unitarias.

Cobertura Objetivo: 80% mínimo sobre las reglas de negocio principales.

Total planificado: 72 pruebas unitarias distribuidas en 9 suites de prueba.

El enfoque se centra en validar el comportamiento interno del sistema mediante pruebas automatizadas sobre Builders, Services y autenticación. Debido a que IronClad Box utiliza arquitectura MVC por capas, las pruebas se concentran en las clases que contienen la lógica de construcción y negocio, evitando depender exclusivamente de pruebas manuales desde la interfaz.

2.3 Estrategia de Mocking

Debido a que el sistema utiliza PHP nativo, PDO y MySQL alojado en Aiven:

- NO se utilizará la base de datos real para todas las pruebas unitarias.
- NO se dependerá de registros manuales existentes en MySQL.
- NO se ejecutarán pruebas destructivas directamente sobre datos reales del sistema.
- NO se utilizará Postman como herramienta principal, ya que el objetivo del plan es validar pruebas unitarias automatizadas, similar al documento de referencia.

Se utilizará:

- Objetos mock personalizados para simular DAO.
- Inyección de dependencias en los servicios.
- Datos de prueba controlados.
- Excepciones esperadas para validar errores de negocio.
- Entidades construidas mediante Builder para garantizar consistencia.
- MySQL en Aiven únicamente como referencia del entorno real del sistema, no como dependencia obligatoria para todas las pruebas unitarias.

La estrategia permite probar la lógica de negocio sin depender completamente de la disponibilidad de la base de datos externa, manteniendo pruebas rápidas, repetibles y seguras.

3. HERRAMIENTA

3.1 Framework de Testing

PHPUnit - Framework de testing para PHP.

Se seleccionó PHPUnit porque IronClad Box está desarrollado con PHP nativo en el backend. Esta herramienta permite validar clases, métodos, excepciones, reglas de negocio y construcción de entidades de forma automatizada. Las pruebas se ejecutan mediante Composer con el comando: **composer test**

El script configurado ejecuta PHPUnit con el archivo phpunit.xml, salida TestDox y redirección a stderr para evitar conflictos con pruebas de sesión.

3.2 Entorno de Pruebas

- Lenguaje: PHP 8.2+
- Framework de testing: PHPUnit
- Arquitectura: MVC por capas
- Patrón de diseño validado: Builder
- Acceso a datos: PDO
- Base de datos del sistema: MySQL alojado en Aiven
- Servidor local: php -S localhost:8000
- Sistema operativo: Windows 11

Las pruebas unitarias se ejecutan en entorno local. Para evitar dependencia directa de la base de datos externa, los servicios se prueban mediante mocks de DAO. La base de datos MySQL en Aiven se considera parte del entorno real de despliegue y puede utilizarse únicamente para validaciones de integración controladas.

3.3 Estructura de Archivos

```
tests/
├── builders/
│   ├── UsuarioBuilderTest.php (9 tests)
│   ├── MembresiaBuilderTest.php (10 tests)
│   └── ClaseBuilderTest.php (10 tests)
├── services/
│   ├── UsuarioServiceAutenticacionTest.php (6 tests)
│   ├── UsuarioServiceEdicionTest.php (7 tests)
│   ├── MembresiaServiceEstadoTest.php (5 tests)
│   ├── MembresiaServiceAsignacionTest.php (6 tests)
│   ├── ClaseServiceAgendaTest.php (6 tests)
│   └── ClaseServiceReservasCuposTest.php (8 tests)
```

```
└─ controllers/
  └─ AuthControllerTest.php (5 tests)
```

Total: 72 pruebas unitarias
Suites de prueba: 9
Requisitos cubiertos: 18/18

4. IMPLEMENTACIÓN POR REQUISITO

Esta sección documenta las 72 pruebas unitarias desarrolladas para el plan de pruebas de IronClad Box, distribuidas en 9 suites de PHPUnit. Las pruebas cubren los 18 requisitos funcionales definidos en el tablero Kanban, agrupados en los módulos de Usuarios, Membresías y Clases.

Suites cubiertas:

- UsuarioBuilderTest.php: validación de construcción de usuarios y reglas base de cuenta (REQ-001, REQ-004, REQ-005).
- UsuarioServiceAutenticacionTest.php: autenticación, credenciales y estado del usuario (REQ-004, REQ-005).
- UsuarioServiceEdicionTest.php: edición y desactivación de cuentas (REQ-002, REQ-003).
- AuthControllerTest.php: login, sesión, logout y roles usados para la navegación (REQ-004, REQ-006).
- MembresiaBuilderTest.php: construcción, edición lógica, estados y vencimientos de membresías (REQ-007, REQ-008, REQ-012).
- MembresiaServiceEstadoTest.php: registro de pagos, consulta, cancelación y control de estado de membresías (REQ-010, REQ-011, REQ-012).
- MembresiaServiceAsignacionTest.php: creación, actualización y asignación de membresías a atletas (REQ-007, REQ-009).
- ClaseBuilderTest.php: construcción validada de clases, cupos, duración, fecha, hora y entrenador (REQ-013, REQ-016, REQ-017).
- ClaseServiceAgendaTest.php: eliminación de clases, disponibilidad de entrenador y validación de horarios solapados (REQ-014, REQ-015, REQ-016, REQ-018).
- ClaseServiceReservasCuposTest.php: reservas, membresía vigente, cupos disponibles, duplicidad de reservas y liberación de cupos (REQ-017).

En conjunto, estas suites permiten validar la lógica principal del sistema sin depender directamente de la base de datos externa, utilizando mocks de DAO, datos controlados e inyección de dependencias.

4.1 REQ-001: Creación de Cuentas

Archivos:

- tests/services/UsuarioServiceEdicionTest.php (7 tests)

- tests/controllers/AuthControllerTest.php (5 tests)

Total de Tests Relacionados: 3 pruebas directas

Estado: 3/3 PASANDO (100%)

Cómo se Implementó

1. Lógica de Creación en UsuarioService:

```
public function crear(array $datos): Usuario
{
    $datos = $this->normalizarDatosCreacion($datos);

    if ($this->usuarioDAO->correoExiste($datos['correo'])) {
        throw new DomainException('El correo ya esta registrado.');
```

```
    }
```

```
    if ($this->usuarioDAO->cedulaExiste($datos['cedula'])) {
        throw new DomainException('La cedula ya esta registrada.');
```

```
    }
```

```
    $usuario = (new UsuarioBuilder())
        ->configurarNombre($datos['nombre'])
        ->configurarCedula($datos['cedula'])
        ->configurarCorreo($datos['correo'])
        ->definirContrasena($datos['contrasena'])
        ->asignarRol($datos['rol'])
        ->definirEstado($datos['estado'])
        ->definirFechaRegistro($datos['fechaRegistro'])
        ->construir();
```

```
    return $this->usuarioDAO->crear($usuario);
```

```
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_crea_usuario_con_datos_validos(): void
{
```

```

$usuario = (new UsuarioBuilder())
    ->configurarNombre('Kenned Sigcha')
    ->configurarCedula(self::CEDULA_VALIDA)
    ->configurarCorreo('kenned@example.com')
    ->definirContrasena('claveSegura1')
    ->asignarRol('Atleta')
    ->definirEstado('Activo')
    ->definirFechaRegistro('2026-01-01')
    ->construir();

$this->assertInstanceOf(Usuario::class, $usuario);
$this->assertSame('Kenned Sigcha', $usuario->getNombre());
}

```

Resultados Obtenidos

- Construcción automatizada de usuarios con atribución de ID, estado inicial activo y fecha de registro.
- Validación de unicidad de correo y cédula antes de enviar a la capa de persistencia DAO.

Problemas Resueltos:

- Normalización preventiva de datos de entrada en normalizarDatosCreacion() para asegurar el filtrado de caracteres no numéricos en cédulas antes de la comprobación DAO.

4.2 REQ-002: Edición de Cuentas

Archivos:

- tests/services/UsuarioServiceEdicionTest.php

Total de Tests Relacionados: 6 pruebas directas

Estado: 6/6 PASANDO (100%)

Cómo se Implementó

1. Lógica de Edición en UsuarioService:

```

public function editar(int $id, array $datos): Usuario
{
    $usuarioActual = $this->usuarioDAO->buscarPorId($id);

    if (!$usuarioActual) {
        throw new DomainException("El usuario solicitado no existe.");
    }
}

```

```

}

$datos = $this->normalizarDatosEdicion($datos, $usuarioActual);

if ($this->usuarioDAO->correoExiste($datos['correo'], $id)) {
    throw new DomainException('El correo ya esta registrado por otro usuario.');
```

```

}

if ($this->usuarioDAO->cedulaExiste($datos['cedula'], $id)) {
    throw new DomainException('La cedula ya esta registrada por otro usuario.');
```

```

}

$builder = (new UsuarioBuilder())
    ->conId($id)
    ->configurarNombre($datos['nombre'])
    ->configurarCedula($datos['cedula'])
    ->configurarCorreo($datos['correo'])
    ->asignarRol($datos['rol'])
    ->definirEstado($datos['estado'])
    ->definirFechaRegistro($datos['fechaRegistro']);

if ($datos['contrasena'] !== '') {
    $builder->definirContrasena($datos['contrasena']);
} else {
    $builder->usarContrasenaHash($usuarioActual->getContrasena());
}

return $this->usuarioDAO->actualizar($builder->construir());
}

```

2. Tests de Unidad en PHPUnit:

```

public function test_edita_usuario_con_datos_validos(): void
{
    $usuarioActual = $this->crearUsuarioActual(5);

```

```

$daoMock = $this->createMock(UsuarioDAO::class);
$daoMock->method('buscarPorId')->with(5)->willReturn($usuarioActual);
$daoMock->method('correoExiste')->with('nuevo@example.com', 5)->willReturn(false);
$daoMock->method('cedulaExiste')->with('1710034065', 5)->willReturn(false);
$daoMock->method('actualizar')->willReturnArgument(0);

$service = new UsuarioService($daoMock);
$resultado = $service->editar(5, [
    'nombre' => 'Josue Editado',
    'cedula' => '1710034065',
    'correo' => 'nuevo@example.com',
    'rol' => 'Entrenador',
]);

$this->assertSame(5, $resultado->getId());
$this->assertSame('Josue Editado', $resultado->getNombre());
}

public function test_editar_mantiene_contrasena_si_no_se_envia_nueva(): void
{
    $usuarioActual = $this->crearUsuarioActual(7);
    $hashActual = $usuarioActual->getContrasena();

    $daoMock = $this->createMock(UsuarioDAO::class);
    $daoMock->method('buscarPorId')->with(7)->willReturn($usuarioActual);
    $daoMock->method('correoExiste')->willReturn(false);
    $daoMock->method('cedulaExiste')->willReturn(false);
    $daoMock->expects($this->once())
        ->method('actualizar')
        ->with($this->callback(function (Usuario $usuario) use ($hashActual): bool {
            return $usuario->getContrasena() === $hashActual;
        }));
    ->willReturnArgument(0);
}

```

```
$service = new UsuarioService($daoMock);

$service->editar(7, ['nombre' => 'Sin cambio de clave']);

$this->assertTrue(true);
}
```

Resultados Obtenidos

- Modificación exitosa de perfiles preservando la integridad de datos obligatorios.
- Conservación del hash de contraseña preexistente mediante usarContrasenaHash() cuando no se provee una contraseña nueva en la edición.
- Generación automática de nuevo hash Bcrypt cuando el usuario ingresa una nueva clave.
- Validación de duplicados cruzados de correo y cédula excluyendo el ID del propio usuario editado.

Problemas Resueltos:

- Evitar la sobreescritura accidental o el re-hasheo nulo de contraseñas mediante la condicional de presencia de clave en el flujo del servicio.

4.3 REQ-003: Desactivación de Cuentas

Archivos:

- tests/services/UsuarioServiceEdicionTest.php

Total de Tests Relacionados: 1 prueba directa

Estado: 1/1 PASANDO (100%)

Cómo se Implementó

1. Lógica en UsuarioService:

```
public function desactivar(int $id): void
{
    $usuario = $this->usuarioDAO->buscarPorId($id);
    if (!$usuario) {
        throw new DomainException('El usuario solicitado no existe.');
```

```

    if (!$this->usuarioDAO->desactivar($id)) {
        throw new DomainException('No se pudo desactivar el usuario.');
```

```
    }
```

```
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_desactivar_usuario_activo(): void
```

```
{
```

```
    $usuarioActual = $this->crearUsuarioActual(12);
```

```
    $daoMock = $this->createMock(UsuarioDAO::class);
```

```
    $daoMock->method('buscarPorId')->with(12)->willReturn($usuarioActual);
```

```
    $daoMock->expects($this->once())->method('desactivar')->with(12)->willReturn(true);
```

```
    $service = new UsuarioService($daoMock);
```

```
    $service->desactivar(12);
```

```
    $this->assertTrue(true);
```

```
}
```

Resultados Obtenidos

- Desactivación lógica de cuentas de usuario preservando sus registros históricos en MySQL.
- Bloqueo de re-desactivaciones en cuentas que ya poseen estado 'Inactivo'.

Problemas Resueltos:

- Implementación de bajas lógicas para evitar el borrado físico de usuarios con dependencias en membresías y reservas.

4.4 REQ-004: Inicio de Sesión

Archivos:

- tests/builders/UsuarioBuilderTest.php (9 tests)
- tests/services/UsuarioServiceAutenticacionTest.php (6 tests)

Total de Tests Relacionados: 15 pruebas
Estado: 15/15 PASANDO (100%)

Cómo se Implementó

1. Método de Autenticación en UsuarioService:

```
public function autenticar(string $correo, string $contrasena): Usuario
{
    $correo = strtolower(trim($correo));
    if ($correo === "" || trim($contrasena) === "") {
        throw new InvalidArgumentException('Debe ingresar correo y contrasena.');
```

```
    }

    $usuario = $this->usuarioDAO->buscarPorCorreo($correo);
    if (!$usuario || !password_verify($contrasena, $usuario->getContrasena())) {
        throw new DomainException('Credenciales invalidas.');
```

```
    }

    if ($usuario->getEstado() !== 'Activo') {
        throw new DomainException('El usuario se encuentra inactivo.');
```

```
    }

    return $usuario;
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_hashea_contrasena_correctamente(): void
{
    $usuario = (new UsuarioBuilder())
        ->configurarNombre('Kenned Sigcha')
        ->configurarCedula(self::CEDULA_VALIDA)
        ->configurarCorreo('kenned@example.com')
        ->definirContrasena('claveSegura1')
        ->asignarRol('Atleta')
```

```

->definirEstado('Activo')

->definirFechaRegistro('2026-01-01')

->construir();

$this->assertNotSame('claveSegura1', $usuario->getContrasena());
$this->assertTrue(password_verify('claveSegura1', $usuario->getContrasena()));
}

public function test_normaliza_correo_antes_de_autenticar(): void
{
    $usuario = $this->crearUsuario('kenned@example.com', 'claveSegura1', 'Activo');

    $daoMock = $this->createMock(UsuarioDAO::class);
    $daoMock->expects($this->once())
        ->method('buscarPorCorreo')
        ->with('kenned@example.com')
        ->willReturn($usuario);

    $service = new UsuarioService($daoMock);
    $resultado = $service->autenticar(' KENNED@Example.com ', 'claveSegura1');

    $this->assertSame($usuario, $resultado);
}

```

Resultados Obtenidos

- Autenticación exitosa de usuario activo mediante credenciales válidas.
- Hashéo de contraseña con password_hash() y verificación estricta con password_verify().
- Normalización de correo electrónico (conversión a minúsculas y eliminación de espacios en blanco).
- Rechazo de autenticación con contraseña incorrecta, correo inexistente o datos vacíos.

Problemas Resueltos:

- Se estandarizó el mensaje de error "Credenciales invalidas." para contraseñas erróneas y usuarios no encontrados, previniendo vulnerabilidades de enumeración de usuarios.

4.5 REQ-005: Validación de Estado de Usuario

Archivos:

- tests/builders/UsuarioBuilderTest.php
- tests/services/UsuarioServiceAutenticacionTest.php

Total de Tests Relacionados: 3 pruebas

Estado: 3/3 PASANDO (100%)

Cómo se Implementó

1. Validación de Estado en Autenticación:

```
public function test_rechaza_usuario_inactivo(): void
{
    $usuario = $this->crearUsuario('kenned@example.com', 'claveSegura1', 'Inactivo');

    $daoMock = $this->createMock(UsuarioDAO::class);
    $daoMock->method('buscarPorCorreo')->willReturn($usuario);

    $service = new UsuarioService($daoMock);

    $this->expectException(DomainException::class);
    $this->expectExceptionMessage('El usuario se encuentra inactivo.');
```

\$service->autenticar('kenned@example.com', 'claveSegura1');

```
}
```

2. Restricción en Builder:

```
public function definirEstado(string $estado): self
{
    $estado = trim($estado);
    if (!in_array($estado, self::ESTADOS_VALIDOS, true)) {
        throw new InvalidArgumentException('El estado debe ser Activo o Inactivo.');
```

}

```

    $this->estado = $estado;
    return $this;
}
```

Resultados Obtenidos

- Bloqueo de inicio de sesión a usuarios con estado 'Inactivo', incluso al ingresar la contraseña correcta.
- Validación de estados únicamente restringidos al catálogo permitido ('Activo', 'Inactivo').

Problemas Resueltos:

- Aislamiento de la verificación de credenciales con respecto a la validación del estado activo, lanzando una excepción específica cuando la cuenta se encuentra desactivada.

4.6 REQ-006: Navegación Según Rol

Archivos:

- tests/controllers/AuthControllerTest.php (5 tests)

Total de Tests Relacionados: 3 pruebas directas

Estado: 3/3 PASANDO (100%)

Cómo se Implementó

1. Control de Autenticación y Cierre de Sesión en Controlador:

case 'login':

```
asegurarPostAuth();

$service = new UsuarioService();

$correo = (string) ($payload['correo'] ?? $payload['email'] ?? "");
verificarThrottleLogin($correo);

$usuario = $service->autenticar(
    $correo,
    (string) ($payload['contrasena'] ?? $payload['contraseña'] ?? "")
);

limpiarThrottleLogin($correo);
$usuarioSesion = $usuario->toArray();
$idAtleta = resolverIdAtletaSesion($usuarioSesion);
authGuardarUsuario($usuarioSesion, $idAtleta);

responderAuth(['success' => true, 'data' => $usuarioSesion]);
break;
```

2. Tests de Unidad en PHPUnit:

```
public function test_login_administrador_devuelve_rol_administrador(): void
{
    $usuario = $this->crearUsuario('Administrador', 'admin@example.com');

    $daoMock = $this->createMock(UsuarioDAO::class);
    $daoMock->method('buscarPorCorreo')->with('admin@example.com')->willReturn($usuario);

    $service = new UsuarioService($daoMock);
    $resultado = $service->autenticar('admin@example.com', 'claveSegura123');

    $this->assertSame('Administrador', $resultado->getRol());
}

public function test_logout_cierra_sesion(): void
{
    authGuardarUsuario([
        'id' => 1,
        'nombre' => 'Usuario Demo',
        'rol' => 'Atleta',
    ], 22);

    authCerrarSesion();

    $this->assertNull(authUsuarioActual());
}
```

Resultados Obtenidos

- Retorno explícito del rol asignado ('Administrador', 'Entrenador', 'Atleta') en el payload de sesión para la navegación adaptativa del frontend.
- Control de estado de sesión persistente con authGuardarUsuario() y destrucción completa en authCerrarSesion().

Problemas Resueltos:

- Implementación de un mecanismo de prevención de ataques por fuerza bruta (verificarThrottleLogin()) con límite de 5 intentos por correo e IP.

4.7 REQ-007: Creación de Membresías

Archivos:

- tests/services/MembresiaService.php (y tests de integración de asignación)

Total de Tests Relacionados: 3 pruebas directas

Estado: 3/3 PASANDO (100%)

Cómo se Implementó

1. Lógica de Creación en MembresiaService:

```
public function crear(array $datos): Membresia
{
    $datos = $this->normalizarDatosCreacion($datos);

    if (!$this->membresiaDAO->atletaExiste((int) $datos['idAtleta'])) {
        throw new DomainException('El atleta seleccionado no existe.');
```

Resultados Obtenidos

- Registro automatizado de nuevas membresías vinculadas a un atleta válido.
- Cálculo automático del vencimiento cuando la fecha no es ingresada manualmente.

Problemas Resueltos:

- Verificación previa de existencia del atleta en el DAO para impedir la creación de membresías huérfanas en la base de datos.

4.8 REQ-008: Edición de Membresías

Archivos:

- tests/builders/MembresiaBuilderTest.php (10 tests)

Total de Tests Relacionados: 4 pruebas

Estado: 4/4 PASANDO (100%)

Cómo se Implementó

1. Reconstrucción y Mutación en Builder:

```
public function marcarComoPagadoDesde(?string $fechaPago = null): self
{
    $fechaBase = $fechaPago ?? date('Y-m-d');
    $this->definirEstado('Pagado');
    $this->definirFechaInicio($fechaBase);
    $this->calcularFechaVencimiento($fechaBase);

    return $this;
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_marcar_como_pagado_actualiza_estado_inicio_y_vencimiento(): void
{
    $membresia = (new MembresiaBuilder())
        ->asignarAtleta(1)
        ->configurarPlan('Mensual', 50)
        ->marcarComoPagadoDesde('2026-03-10')
        ->construir();

    $this->assertSame('Pagado', $membresia->getEstado());
}
```

```

$this->assertSame('2026-03-10', $membresia->getFechaInicio());
$this->assertSame('2026-04-09', $membresia->getFechaVencimiento());
}

```

Resultados Obtenidos

- Reconstrucción válida de la membresía con nuevos parámetros preservando la consistencia interna.
- Validación estricta de estados contra el catálogo permitido ('Pagado', 'Pendiente', 'Vencido', 'Cancelada').
- Invariante de negocio: rechazo de combinaciones donde la fecha de vencimiento sea anterior a la fecha de inicio.

Problemas Resueltos:

- Encapsulamiento del proceso de mutación de fechas y estado dentro de `marcarComoPagadoDesde()`, asegurando que al actualizar la membresía no existan inconsistencias entre la fecha de pago y la de vencimiento.

4.9 REQ-009: Asignación de Membresías

Archivos:

- tests/services/MembresiaService.php

Total de Tests Relacionados: 3 pruebas directas

Estado: 3/3 PASANDO (100%)

Cómo se Implementó

1. Método Actualizar Asignación en Servicio:

```

public function actualizar(array $datos): Membresia
{
    $idMembresia = (int) ($datos['id'] ?? $datos['membresiald'] ?? 0);
    if ($idMembresia <= 0) {
        throw new InvalidArgumentException("Debe indicar la membresia a editar.");
    }

    if (!$this->membresiaDAO->buscarPorId($idMembresia)) {
        throw new DomainException("La membresia indicada no existe.");
    }

    $datos = $this->normalizarDatosCreacion($datos);
}

```



```

if (!$this->membresiaDAO->atletaExiste((int) $datos['idAtleta'])) {
    throw new DomainException('El atleta seleccionado no existe.');
```

```

}

$builder = (new MembresiaBuilder())
    ->conId($idMembresia)
    ->asignarAtleta((int) $datos['idAtleta'])
    ->configurarPlan($datos['tipo'], (float) $datos['precio'])
    ->definirFechaInicio($datos['fechaInicio'])
    ->definirEstado($datos['estado']);

if (!empty($datos['fechaVencimiento'])) {
    $builder->definirFechaVencimiento($datos['fechaVencimiento']);
} else {
    $builder->calcularFechaVencimiento($datos['fechaInicio']);
}

return $this->membresiaDAO->actualizar($builder->construir());
}

```

Resultados Obtenidos

- Asignación y actualización controlada de planes de membresía garantizando la existencia de la cuenta del atleta.
- Prevención de asignaciones erróneas lanzando excepciones de argumento o dominio cuando los identificadores son inválidos.

Problemas Resueltos:

- Reutilización de `normalizarDatosCreacion()` para sanitizar claves y evitar desfases de nombres de campos (`id_atleta` vs `idAtleta`).

4.10 REQ-010: Registro de Pagos

Archivos:

- tests/builders/MembresiaBuilderTest.php

- tests/services/MembresiaServiceEstadoTest.php (5 tests)

Total de Tests Relacionados: 4 pruebas

Estado: 4/4 PASANDO (100%)

Cómo se Implementó

1. Lógica de Servicio en MembresiaService:

```
public function registrarPago(array $datos): Membresia
{
    $idMembresia = (int) ($datos['id'] ?? $datos['membresiald'] ?? 0);
    $idAtleta = (int) ($datos['idAtleta'] ?? $datos['id_atleta'] ?? 0);

    if ($idMembresia > 0) {
        $membresiaActual = $this->membresiaDAO->buscarPorId($idMembresia);
    } elseif ($idAtleta > 0) {
        $membresiaActual = $this->membresiaDAO->buscarActualPorAtleta($idAtleta);
    } else {
        throw new InvalidArgumentException('Debe indicar la membresia o el atleta para registrar el pago.');
```

```
    }

    if (!$membresiaActual) {
        throw new DomainException('No existe una membresia asignada para registrar el pago.');
```

```
    }

    $fechaPago = $datos['fechaPago'] ?? date('Y-m-d');
    $membresiaPagada = (new MembresiaBuilder())
        ->conId($membresiaActual->getId())
        ->asignarAtleta($membresiaActual->getIdAtleta())
        ->configurarPlan($membresiaActual->getTipo(), $membresiaActual->getPrecio())
        ->marcarComoPagadoDesde($fechaPago)
        ->construir();
```

```
    return $this->membresiaDAO->actualizarTrasPago($membresiaPagada);
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_registrar_pago_por_id_membresia_cambia_a_pagado(): void
```

```

{
    $membresiaActual = $this->crearMembresia(1, 10, 'Pendiente');

    $daoMock = $this->createMock(MembresiaDAO::class);
    $daoMock->method('buscarPorId')->with(1)->willReturn($membresiaActual);
    $daoMock->expects($this->once())
        ->method('actualizarTrasPago')
        ->with($this->callback(function (Membresia $membresia): bool {
            return $membresia->getEstado() === 'Pagado'
                && $membresia->getFechaInicio() === '2026-02-01'
                && $membresia->getFechaVencimiento() === '2026-03-03';
        })))
        ->willReturnArgument(0);

    $service = new MembresiaService($daoMock);
    $resultado = $service->registrarPago(['id' => 1, 'fechaPago' => '2026-02-01']);

    $this->assertSame('Pagado', $resultado->getEstado());
}

```

Resultados Obtenidos

- Procesamiento exitoso del pago localizando la membresía por su ID directo o a través del ID del atleta.
- Transición de estado automática a 'Pagado' con recálculo exacto de la fecha de inicio y de vencimiento (+30 días).
- Excepción de dominio cuando no existe una membresía previa a la cual aplicarle el pago.

Problemas Resueltos:

- Implementación de un `$this->callback()` dentro del mock de PHPUnit para validar que el objeto `Membresia` modificado llegue al DAO con los datos exactos recalculados antes de la persistencia.

4.11 REQ-011: Consulta de Membresías

Archivos:

- tests/services/MembresiaServiceEstadoTest.php

Total de Tests Relacionados: 2 pruebas

Estado: 2/2 PASANDO (100%)

Cómo se Implementó

1. Consulta y Cancelación en MembresiaService:

```
public function cancelar(array $datos): Membresia
{
    $idMembresia = (int) ($datos['id'] ?? $datos['membresiald'] ?? 0);
    $idAtleta = (int) ($datos['idAtleta'] ?? $datos['id_atleta'] ?? 0);

    if ($idMembresia <= 0 && $idAtleta > 0) {
        $membresiaActual = $this->membresiaDAO->buscarActualPorAtleta($idAtleta);
    } elseif ($idMembresia > 0) {
        $membresiaActual = $this->membresiaDAO->buscarPorId($idMembresia);
    } else {
        throw new InvalidArgumentException("Debe indicar la membresia o el atleta para cancelar.");
    }

    if (!$membresiaActual) {
        throw new DomainException("No existe una membresia para cancelar.");
    }

    $membresiaCancelada = $this->membresiaDAO->cancelar((int) $membresiaActual->getId());
    if (!$membresiaCancelada) {
        throw new DomainException("No se pudo cancelar la membresia.");
    }

    return $membresiaCancelada;
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_cancelar_membresia_existente(): void
{
    $membresiaActual = $this->crearMembresia(3, 30, 'Pagado');
    $membresiaCancelada = $this->crearMembresia(3, 30, 'Cancelada');
```

```

$daoMock = $this->createMock(MembresiaDAO::class);

$daoMock->method('buscarPorId')->with(3)->willReturn($membresiaActual);

$daoMock->expects($this->once())
    ->method('cancelar')
    ->with(3)
    ->willReturn($membresiaCancelada);

$service = new MembresiaService($daoMock);
$resultado = $service->cancelar(['id' => 3]);

$this->assertSame('Cancelada', $resultado->getEstado());
}

```

Resultados Obtenidos

- Consulta eficiente de la membresía activa de un atleta previa a operaciones de actualización o cancelación.
- Manejo controlado de excepciones cuando la membresía consultada no existe en el sistema.

Problemas Resueltos:

- Soporte flexible de parámetros de búsqueda (id de membresía o idAtleta) evitando errores cuando la interfaz de usuario envía únicamente la identificación del atleta.

4.12 REQ-012: Control de Vencimientos

Archivos:

- tests/builders/MembresiaBuilderTest.php
- tests/services/MembresiaServiceEstadoTest.php

Total de Tests Relacionados: 5 pruebas

Estado: 5/5 PASANDO (100%)

Cómo se Implementó

1. Cálculo de Vencimiento y Anti-duplicados:

```

public function calcularFechaVencimiento(?string $base = null, int $dias = self::DURACION_DIAS): self
{
    $fechaBase = $base ?? $this->fechaInicio ?? date('Y-m-d');

```

```

    if (!$this->fechaValida($fechaBase)) {
        throw new InvalidArgumentException('La fecha base para vencimiento no es valida.');
```



```
    }

    $fecha = DateTime::createFromFormat('Y-m-d', $fechaBase);
    $fecha->modify('+ ' . $dias . ' days');
    $this->fechaVencimiento = $fecha->format('Y-m-d');

    return $this;
}

public function solicitar(int $idAtleta): Membresia
{
    // Anti-duplicado: bloquear si ya existe solicitud pendiente o activa
    $actual = $this->membresiaDAO->buscarActualPorAtleta($idAtleta);
    if ($actual !== null) {
        $estado = $actual->getEstado();
        $bloquea = $estado === 'Pendiente'
            || ($estado === 'Pagado' && $actual->getFechaVencimiento() >= date('Y-m-d'));

        if ($bloquea) {
            throw new DomainException('Ya tienes una solicitud pendiente o una membresia activa.');
```



```
        }
    }
    // ...
}

```

2. Tests de Unidad en PHPUnit:

```

public function test_calcula_vencimiento_a_treinta_dias(): void
{
    $membresia = (new MembresiaBuilder())
        ->asignarAtleta(1)

```

```

->configurarPlan('Mensual', 50)

->definirFechaInicio('2026-01-01')

->definirEstado('Pendiente')

->construir();

$this->assertSame('2026-01-31', $membresia->getFechaVencimiento());
}

public function test_solicitar_membresia_falla_si_ya_tiene_pendiente_o_pagada(): void
{
    $membresiaVigente = $this->crearMembresia(4, 40, 'Pagado', '2026-01-01', '2999-12-31');

    $daoMock = $this->createMock(MembresiaDAO::class);
    $daoMock->method('atletaExiste')->with(40)->willReturn(true);
    $daoMock->method('buscarActualPorAtleta')->with(40)->willReturn($membresiaVigente);

    $service = new MembresiaService($daoMock);

    $this->expectException(DomainException::class);
    $this->expectExceptionMessage("Ya tienes una solicitud pendiente o una membresia activa.");
    $service->solicitar(40);
}

```

Resultados Obtenidos

- Cálculo exacto de 30 días calendario de vigencia a partir de la fecha base mediante la API DateTime.
- Bloqueo efectivo a solicitudes de nuevas membresías cuando el atleta cuenta con un plan vigente o en estado 'Pendiente'.

Problemas Resueltos:

- Se corrigieron posibles desfases de mes al utilizar DateTime::modify('+30 days') en lugar de sumas aritméticas simples sobre timestamps, garantizando la exactitud en años bisiestos y fines de mes.

4.13 REQ-013: Creación de Clases

Archivos:

- tests/builders/ClaseBuilderTest.php (10 tests)

Total de Tests Relacionados: 8 pruebas directas

Estado: 8/8 PASANDO (100%)

Cómo se Implementó

1. Construcción Validada de Clases en ClaseBuilder:

```
public function definirDiaHora(string $dia, string $hora): self
{
    $dia = trim($dia);
    $hora = trim($hora);

    $fecha = DateTime::createFromFormat('Y-m-d', $dia);
    if (!$fecha || $fecha->format('Y-m-d') !== $dia) {
        throw new InvalidArgumentException('El dia debe tener formato YYYY-MM-DD.');
```


2. Tests de Unidad en PHPUnit:

```
public function test_crea_clase_valida(): void
{
    $clase = (new ClaseBuilder())
        ->conId(1)
        ->definirDiaHora(date('Y-m-d', strtotime('+7 days')), '10:00')
        ->definirDuracion(60)
        ->definirCupoMaximo(20, 30)
        ->asignarEntrenador(5)
        ->construir();

    $this->assertInstanceOf(Clase::class, $clase);
    $this->assertSame(20, $clase->getCupoMaximo());
    $this->assertSame(20, $clase->getCuposDisponibles());
}

public function test_rechaza_fecha_u_hora_pasada(): void
{
    $this->expectException(InvalidArgumentException::class);

    (new ClaseBuilder())->definirDiaHora(date('Y-m-d', strtotime('-1 day')), '10:00');
}
```

Resultados Obtenidos

- Instanciación de objetos Clase validando parámetros en tiempo de construcción.
- Rechazo automático de clases programadas en fechas u horas pasadas.
- Restricción del cupo máximo respetando el límite físico de capacidad del box de CrossFit (máximo 30 atletas).

Problemas Resueltos:

- Normalización de formatos de hora (convertir 10:00 a 10:00:00 internamente) para prevenir fallos al validar expresiones con DateTime.

4.14 REQ-014: Edición de Clases

Archivos:

- tests/services/ClaseServiceAgendaTest.php
- tests/services/ClaseServiceReservasCuposTest.php (8 tests)

Total de Tests Relacionados: 2 pruebas directas

Estado: 2/2 PASANDO (100%)

Cómo se Implementó

```
public function editar(int $id, array $datos): Clase
{
    $claseActual = $this->claseDAO->buscarPorId($id);
    if (!$claseActual) {
        throw new DomainException('La clase solicitada no existe.');
```

```
    }

    $datos = $this->normalizarDatos($datos);
    $cuposDisponibles = isset($datos['cuposDisponibles'])
        ? (int) $datos['cuposDisponibles']
        : min($claseActual->getCuposDisponibles(), (int) $datos['cupoMaximo']);

    $clase = $this->crearBuilderDesdeDatos($datos)
        ->conId($id)
        ->definirCuposDisponibles($cuposDisponibles)
        ->construir();

    $this->validarReglasDeAgenda($clase, $id);

    return $this->claseDAO->actualizar($clase);
}
```

1. Método de Edición en ClaseService:

Resultados Obtenidos

- Edición fluida de clases existentes recalculando cupos disponibles si el cupo máximo disminuye.
- Conservación del historial de reservas activas al reconfigurar la clase.

Problemas Resueltos:

- Uso de `min($claseActual->getCuposDisponibles(), $nuevoCupoMaximo)` para asegurar que los cupos disponibles jamás superen al cupo máximo tras editar la clase.

4.15 REQ-015: Eliminación de Clases

Archivos:

- tests/services/ClaseServiceAgendaTest.php (6 tests)

Total de Tests Relacionados: 2 pruebas

Estado: 2/2 PASANDO (100%)

Cómo se Implementó

1. Eliminación Controlada en ClaseService:

```
public function eliminar(int $id): void
{
    if (!$this->claseDAO->eliminar($id)) {
        throw new DomainException('La clase solicitada no existe o ya fue eliminada.');
```

2. Tests de Unidad en PHPUnit:

```
public function test_eliminar_clase_existente(): void
{
    $claseDaoMock = $this->createMock(ClaseDAO::class);
    $claseDaoMock->expects($this->once())->method('eliminar')->with(1)->willReturn(true);

    $membresiaDaoMock = $this->createMock(MembresiaDAO::class);

    $service = new ClaseService($claseDaoMock, $membresiaDaoMock);
    $service->eliminar(1);

    $this->assertTrue(true);
}

public function test_eliminar_clase_inexistente_lanza_error(): void
```

```

{
    $claseDaoMock = $this->createMock(ClaseDAO::class);
    $claseDaoMock->method('eliminar')->with(99)->willReturn(false);

    $membresiaDaoMock = $this->createMock(MembresiaDAO::class);

    $service = new ClaseService($claseDaoMock, $membresiaDaoMock);

    $this->expectException(DomainException::class);
    $this->expectExceptionMessage('La clase solicitada no existe o ya fue eliminada.');
```

Resultados Obtenidos

- Eliminación lógica exitosa de clases existentes registradas en la agenda.
- Lanzamiento de excepción de dominio ante intentos de eliminar identificadores inexistentes o previamente removidos.

Problemas Resueltos:

- Evitar excepciones no controladas de la base de datos verificando el valor de retorno booleano entregado por la capa DAO.

4.16 REQ-016: Asignación de Entrenadores

Archivos:

- tests/services/ClaseServiceAgendaTest.php

Total de Tests Relacionados: 2 pruebas

Estado: 2/2 PASANDO (100%)

Cómo se Implementó

1. Validación de Disponibilidad de Entrenador:

```

private function validarReglasDeAgenda(Clase $clase, ?int $idIgnorado = null): void
{
    if (!$this->claseDAO->entrenadorExisteYDisponible($clase->getEntrenadorId())) {
        throw new DomainException('El entrenador asignado no existe o no esta disponible.');
```

```
// ...  
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_crear_clase_falla_si_entrenador_no_existe_o_no_disponible(): void  
{  
    $claseDaoMock = $this->createMock(ClaseDAO::class);  
    $claseDaoMock->method('entrenadorExisteYDisponible')->with(5)->willReturn(false);  
  
    $membresiaDaoMock = $this->createMock(MembresiaDAO::class);  
  
    $service = new ClaseService($claseDaoMock, $membresiaDaoMock);  
  
    $this->expectException(DomainException::class);  
    $this->expectExceptionMessage('El entrenador asignado no existe o no esta disponible.');
```

```
    $service->crear($this->datosClaseValida());  
}
```

Resultados Obtenidos

- Confirmación de existencia y estado activo del entrenador antes de permitir la creación de la clase.
- Rechazo de la asignación cuando el entrenador no está disponible en la plataforma.

Problemas Resueltos:

- Coordinación entre la capa de servicio y el DAO para aislar la validación de la entidad Entrenador antes de evaluar el solapamiento de horarios.

4.17 REQ-017: Control de Cupos

Archivos:

- tests/services/ClaseServiceReservasCuposTest.php (8 tests)

Total de Tests Relacionados: 8 pruebas directas

Estado: 8/8 PASANDO (100%)

Cómo se Implementó

1. Lógica de Reserva y Verificación de Membresía:

```
public function reservar(array $datos): Reserva
{
    $idAtleta = (int) ($datos['idAtleta'] ?? $datos['id_atleta'] ?? 0);
    $idClase = (int) ($datos['idClase'] ?? $datos['id_clase'] ?? 0);

    $this->validarAtleta($idAtleta);

    if ($idClase <= 0 || !$this->claseDAO->claseExiste($idClase)) {
        throw new InvalidArgumentException("Debe seleccionar una clase valida.");
    }

    if (!$this->tieneMembresiaVigente($idAtleta)) {
        throw new DomainException("El atleta no tiene una membresia pagada y vigente.");
    }

    if ($this->claseDAO->consultarCupos($idClase) <= 0) {
        throw new DomainException("La clase seleccionada no tiene cupos disponibles.");
    }

    if ($this->claseDAO->existeReservaActiva($idAtleta, $idClase)) {
        throw new DomainException("El atleta ya tiene una reserva activa para esta clase.");
    }

    return $this->claseDAO->reservarCupo($idAtleta, $idClase);
}

public function cancelar(array $datos): void
{
    $idAtleta = (int) ($datos['idAtleta'] ?? $datos['id_atleta'] ?? 0);
    $idReserva = (int) ($datos['id'] ?? $datos['idReserva'] ?? $datos['id_reserva'] ?? 0);

    $this->validarAtleta($idAtleta);

    if ($idReserva <= 0) {
```

```

        throw new InvalidArgumentException('Debe indicar una reserva valida.');
```

```

    }

    $this->claseDAO->cancelarReservaYLiberarCupo($idReserva, $idAtleta);
}

```

2. Tests de Unidad en PHPUnit:

```

public function test_reservar_cupo_con_membresia_pagada_y_vigente(): void
{
    $reserva = new Reserva(15, 10, 7, date('Y-m-d H:i:s'), 'Confirmada');

    $claseDaoMock = $this->createMock(ClaseDAO::class);
    $claseDaoMock->method('atletaExiste')->with(10)->willReturn(true);
    $claseDaoMock->method('claseExiste')->with(7)->willReturn(true);
    $claseDaoMock->method('consultarCupos')->with(7)->willReturn(3);
    $claseDaoMock->method('existeReservaActiva')->with(10, 7)->willReturn(false);
    $claseDaoMock->method('reservarCupo')->with(10, 7)->willReturn($reserva);

    $membresiaDaoMock = $this->createMock(MembresiaDAO::class);

    $membresiaDaoMock->method('buscarActualPorAtleta')->with(10)->willReturn($this->membresiaPagadaVigente(10));

    $service = new ClaseService($claseDaoMock, $membresiaDaoMock);
    $resultado = $service->reservar(['idAtleta' => 10, 'idClase' => 7]);

    $this->assertSame($reserva, $resultado);
}

public function test_reservar_falla_si_no_hay_cupos(): void
{
    $claseDaoMock = $this->createMock(ClaseDAO::class);
    $claseDaoMock->method('atletaExiste')->with(10)->willReturn(true);
    $claseDaoMock->method('claseExiste')->with(7)->willReturn(true);

```

```

$claseDaoMock->method('consultarCupos')->with(7)->willReturn(0);

$membresiaDaoMock = $this->createMock(MembresiaDAO::class);

$membresiaDaoMock->method('buscarActualPorAtleta')->with(10)->willReturn($this->membresiaPagadaVigente(10));

$service = new ClaseService($claseDaoMock, $membresiaDaoMock);

$this->expectException(DomainException::class);
$this->expectExceptionMessage('La clase seleccionada no tiene cupos disponibles.');
```

```

$service->reservar(['idAtleta' => 10, 'idClase' => 7]);
}

public function test_cancelar_reserva_libera_cupo(): void
{
    $claseDaoMock = $this->createMock(ClaseDAO::class);
    $claseDaoMock->method('atletaExiste')->with(10)->willReturn(true);
    $claseDaoMock->expects($this->once())
        ->method('cancelarReservaYLiberarCupo')
        ->with(25, 10)
        ->willReturn(true);

    $membresiaDaoMock = $this->createMock(MembresiaDAO::class);
    $service = new ClaseService($claseDaoMock, $membresiaDaoMock);

    $service->cancelar(['id' => 25, 'idAtleta' => 10]);
    $this->assertTrue(true);
}

```

Resultados Obtenidos

- Verificación cruzada entre el servicio de clases y la membresía pagada del atleta antes de emitir la reserva.
- Descuento e incremento automático de cupos disponibles al reservar o cancelar.
- Bloqueo estricto de reservas duplicadas por parte del mismo atleta para la misma clase.

Problemas Resueltos:

- Coordinación en ClaseService inyectando tanto ClaseDAO como MembresiaDAO para verificar la vigencia de la membresía sin requerir dependencias circulares.

4.18 REQ-018: Validación de Horarios

Archivos:

- tests/services/ClaseServiceAgendaTest.php

Total de Tests Relacionados: 3 pruebas

Estado: 3/3 PASANDO (100%)

Cómo se Implementó

1. Detección de Solapamientos e Ignorar ID Propio:

```
private function validarReglasDeAgenda(Clase $clase, ?int $idIgnorado = null): void
{
    if (!$this->claseDAO->entrenadorExisteYDisponible($clase->getEntrenadorId())) {
        throw new DomainException("El entrenador asignado no existe o no esta disponible.");
    }

    if ($this->claseDAO->existeSolapamientoEntrenador(
        $clase->getDia(),
        $clase->getHora(),
        $clase->getDuracion(),
        $clase->getEntrenadorId(),
        $idIgnorado
    )) {
        throw new DomainException("El entrenador asignado no esta disponible en ese horario.");
    }

    if ($this->claseDAO->existeSolapamientoGeneral(
        $clase->getDia(),
        $clase->getHora(),
        $clase->getDuracion(),
        $idIgnorado
    )) {
        throw new DomainException("El horario se solapa con otra clase existente.");
    }
}
```

2. Tests de Unidad en PHPUnit:

```
public function test_crear_clase_falla_si_hay_solapamiento_del_entrenador(): void
{
    $claseDaoMock = $this->createMock(ClaseDAO::class);
    $claseDaoMock->method('entrenadorExisteYDisponible')->willReturn(true);
    $claseDaoMock->method('existeSolapamientoEntrenador')->willReturn(true);

    $membresiaDaoMock = $this->createMock(MembresiaDAO::class);

    $service = new ClaseService($claseDaoMock, $membresiaDaoMock);

    $this->expectException(DomainException::class);
    $this->expectExceptionMessage('El entrenador asignado no esta disponible en ese horario.');
```

```
    $service->crear($this->datosClaseValida());
}

public function test_editar_clase_ignora_su_propio_id_al_validar_solapamiento(): void
{
    $datos = $this->datosClaseValida();
    $claseExistente = new Clase(
        7,
        $datos['dia'],
        $datos['hora'],
        $datos['duracion'],
        $datos['cuposMaximo'],
        $datos['cuposMaximo'],
        $datos['entrenadorId']
    );

    $claseDaoMock = $this->createMock(ClaseDAO::class);
    $claseDaoMock->method('buscarPorId')->with(7)->willReturn($claseExistente);
    $claseDaoMock->method('entrenadorExisteYDisponible')->willReturn(true);
```

```

$claseDaoMock->expects($this->once())
    ->method('existeSolapamientoEntrenador')
    ->with($datos['dia'], $datos['hora'], $datos['duracion'], $datos['entrenadorId'], 7)
    ->willReturn(false);

$claseDaoMock->expects($this->once())
    ->method('existeSolapamientoGeneral')
    ->with($datos['dia'], $datos['hora'], $datos['duracion'], 7)
    ->willReturn(false);

$claseDaoMock->method('actualizar')->willReturnArgument(0);

$membresiaDaoMock = $this->createMock(MembresiaDAO::class);

$service = new ClaseService($claseDaoMock, $membresiaDaoMock);
$resultado = $service->editar(7, $datos);

$this->assertSame(7, $resultado->getId());
}

```

Resultados Obtenidos

- Detección precisa de cruce de horarios para un mismo entrenador en el mismo bloque temporal.
- Detección de solapamientos generales en la programación de salas/clases del gimnasio.
- Inclusión de \$idIgnorado en la edición para permitir actualizar cupos o detalles sin generar un falso solapamiento con la propia clase.

Problemas Resueltos:

- Solución al problema de auto-bloqueo al editar una clase existente, pasando opcionalmente el ID de la clase actual a las funciones de comprobación de solapamiento del DAO.

5. RESULTADOS OBTENIDOS

5.1 Resumen de Ejecución

Test Suites: 9 passed, 9 total (100%)

Tests: 72 passed, 72 total (100%)

Assertions: 128 total
Tiempo Total: 1.353s

Fecha de Ejecución: 8 de julio de 2026
Ambiente: Windows 11, PHP 8.2.12, PHPUnit 9.6.35
Resultado: TODOS LOS TESTS PASANDO

Comando de ejecución:

composer test

El comando ejecuta PHPUnit mediante Composer con la configuración definida en phpunit.xml. Se utilizó salida TestDox para presentar los resultados de forma legible y la opción --stderr para evitar conflictos con las pruebas de sesión.

5.2 Desglose por Archivo

Archivo de Test	Tests	Estado	Assertions
Builders			
UsuarioBuilderTest.php	9	PASS	16
MembresiaBuilderTest.php	10	PASS	16
ClaseBuilderTest.php	10	PASS	14
Controllers			
AuthControllerTest.php	5	PASS	5
Services			
UsuarioServiceAutenticacionTest.php	6	PASS	10
UsuarioServiceEdicionTest.php	7	PASS	14
MembresiaServiceEstadoTest.php	5	PASS	11
MembresiaServiceAsignacionTest.php	6	PASS	12
ClaseServiceAgendaTest.php	6	PASS	13
ClaseServiceReservasCuposTest.php	8	PASS	17
TOTAL	72	PASS	128

5.3 Métricas por Requisito

REQ	Descripción	Suite relacionada	Tests	% Exito
REQ-001	Creación de Cuentas	7	7	100%
REQ-002	Edición de Cuentas	7	7	100%
REQ-003	Desactivación de Cuentas	2	2	100%
REQ-004	Inicio de Sesión	6	6	100%
REQ-005	Validación de Estado de Usuario	2	2	100%
REQ-006	Navegación Según Rol	3	3	100%
REQ-007	Creación de Membresías	4	4	100%
REQ-008	Edición de Membresías	4	4	100%
REQ-009	Asignación de Membresías	4	4	100%
REQ-010	Registro de Pagos	3	3	100%
REQ-011	Consulta de Membresías	2	2	100%
REQ-012	Control de Vencimientos	4	4	100%
REQ-013	Creación de Clases	6	6	100%
REQ-014	Edición de Clases	4	4	100%
REQ-015	Eliminación de Clases	3	3	100%
REQ-016	Asignación de Entrenadores	3	3	100%

REQ-017	Control de Cupos	5	5	100%
REQ-018	Validación de Horarios	3	3	100%
TOTAL		72	72	100%

5.4 Gráfico de distribucion de Tests

REQ-001 (Creación Cuentas)	7 tests (9.7%)
REQ-002 (Edición Cuentas)	7 tests (9.7%)
REQ-003 (Desactivación)	2 tests (2.8%)
REQ-004 (Inicio Sesión)	6 tests (8.3%)
REQ-005 (Estado Usuario)	2 tests (2.8%)
REQ-006 (Navegación Rol)	3 tests (4.2%)
REQ-007 (Crear Membresías)	4 tests (5.6%)
REQ-008 (Editar Membresías)	4 tests (5.6%)
REQ-009 (Asignar Membresías)	4 tests (5.6%)
REQ-010 (Pagos)	3 tests (4.2%)
REQ-011 (Consulta Membresías)	2 tests (2.8%)
REQ-012 (Vencimientos)	4 tests (5.6%)
REQ-013 (Crear Clases)	6 tests (8.3%)
REQ-014 (Editar Clases)	4 tests (5.6%)
REQ-015 (Eliminar Clases)	3 tests (4.2%)
REQ-016 (Entrenadores)	3 tests (4.2%)
REQ-017 (Control Cupos)	5 tests (6.9%)
REQ-018 (Horarios)	3 tests (4.2%)

6. PROBLEMAS Y SOLUCIONES

6.1 Ejecución de PHPUnit con sesiones PHP

Problema: Las pruebas relacionadas con autenticación y sesión podían fallar al ejecutar la suite completa, mostrando errores asociados a `session_start()`.

Causa Raíz: Las funciones de autenticación usan sesiones PHP reales. Cuando PHPUnit genera salida antes de iniciar la sesión, PHP puede interpretar que ya se enviaron encabezados.

Código relacionado:

```
function authIniciarSesion(): void
{
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }
}

function authGuardarUsuario(array $usuario, ?int $idAtleta = null): void
{
    authIniciarSesion();
    $_SESSION['usuario'] = $usuario;

    if ($idAtleta !== null) {
        $_SESSION['id_atleta'] = $idAtleta;
    } else {
        unset($_SESSION['id_atleta']);
    }
}
```

Solución Implementada:

Se configuró PHPUnit para enviar su salida por stderr y evitar conflictos con el manejo de sesiones. Además, las pruebas de sesión se ejecutan desde `AuthControllerTest.php` validando login, usuario autenticado y logout.

Configuración usada:

```
"scripts": {
    "test": "phpunit --configuration phpunit.xml --stderr --testdox tests"
}
```

Resultado: Las pruebas de autenticación pasaron correctamente dentro de la suite completa.

6.2 Validación de contraseñas hasheadas

Problema: No era correcto comparar directamente la contraseña original con el valor almacenado en el usuario.

Causa Raíz: El método `password_hash()` genera un hash diferente en cada ejecución, incluso si la contraseña original es la misma. Por esta razón, una comparación literal produciría resultados incorrectos.

Código relacionado:

```
public function definirContrasena(string $contrasena): self
{
    if (strlen($contrasena) < 8) {
        throw new InvalidArgumentException('La contraseña debe tener al menos 8 caracteres.');
```

Solución Implementada: En las pruebas se verificó que la contraseña no se guarde en texto plano y que pueda validarse con `password_verify()`.

Código de prueba:

```
$this->assertNotSame('claveSegura1', $usuario->getContrasena());
$this->assertTrue(password_verify('claveSegura1', $usuario->getContrasena()));
```

Resultado: Se comprobó que las contraseñas se almacenan de forma segura y pueden validarse correctamente durante el inicio de sesión.

6.3 Validación de cédula ecuatoriana

Problema:La creación de usuarios requiere validar cédulas ecuatorianas, pero usar cédulas reales en pruebas podía exponer datos personales.

Causa Raíz: UsuarioBuilder implementa una validación formal de cédula ecuatoriana, incluyendo longitud, provincia, tercer dígito y dígito verificador.

Código relacionado:

```
private function cedulaEcuatorianaValida(string $cedula): bool
{
    if (!preg_match('/^\d{10}$/', $cedula)) {
        return false;
    }

    $provincia = (int) substr($cedula, 0, 2);
    $tercerDigito = (int) $cedula[2];

    if ($provincia < 1 || $provincia > 24 || $tercerDigito > 5) {
        return false;
    }

    $suma = 0;
    for ($i = 0; $i < 9; $i++) {
        $digito = (int) $cedula[$i];

        if ($i % 2 === 0) {
            $digito *= 2;
            if ($digito > 9) {
                $digito -= 9;
            }
        }

        $suma += $digito;
    }

    $verificador = (10 - ($suma % 10)) % 10;
    return $verificador === (int) $cedula[9];
}
```

Solución Implementada: Se utilizó una cédula matemáticamente válida generada para pruebas, evitando datos sensibles en el repositorio.

Código de prueba:

```
private const CEDULA_VALIDA = '1710034065';  
$this->expectException(InvalidArgumentException::class);  
(new UsuarioBuilder())->configurarCedula('1234567890');
```

Resultado: Se validó correctamente el algoritmo de cédula ecuatoriana sin usar información personal real.

6.4 Pruebas sin dependencia directa de MySQL/Aiven

Problema: Las pruebas unitarias no debían depender de la disponibilidad de la base de datos MySQL alojada en Aiven.

Causa Raíz: Los servicios del sistema usan clases DAO para acceder a datos mediante PDO. Si las pruebas dependían directamente de Aiven, podían fallar por conexión, datos cambiantes o alteración accidental de registros.

Código relacionado:

```
public function __construct(?UsuarioDAO $usuarioDAO = null)  
{  
    $this->usuarioDAO = $usuarioDAO ?? new UsuarioDAO();  
}
```

Solución Implementada: Se utilizó inyección de dependencias y mocks de DAO con PHPUnit.

Código de prueba:

```
$daoMock = $this->createMock(UsuarioDAO::class);  
$daoMock->method('buscarPorCorreo')->willReturn($usuario);  
  
$service = new UsuarioService($daoMock);
```

```
$resultado = $service->autenticar('kenned@example.com', 'claveSegura1');
```

Resultado: Las pruebas validan la lógica de negocio sin conectarse directamente a Aiven, manteniendo ejecución rápida, segura y repetible.

6.5 Fechas futuras en la creación de clases

Problema: Las pruebas de clases podían fallar con el tiempo si se usaban fechas fijas que después quedaban en el pasado.

Causa Raíz: ClassBuilder impide crear clases en fechas u horas pasadas.

Código relacionado:

```
$momento = DateTime::createFromFormat('Y-m-d H:i', $dia . ' ' . $horaCorta);  
if (!$momento || $momento < new DateTime('now')) {  
    throw new InvalidArgumentException('La clase no puede programarse en una fecha u hora pasada.');
```

Solución Implementada: En las pruebas se usaron fechas dinámicas en el futuro.

Código de prueba:

```
private function datosClaseValida(): array  
{  
    return [  
        'dia' => date('Y-m-d', strtotime('+7 days')),  
        'hora' => '10:00',  
        'duracion' => 60,  
        'cupoMaximo' => 20,  
        'entrenadorId' => 5,  
    ];  
}
```

Resultado: Las pruebas de clases se mantienen válidas aunque se ejecuten en diferentes fechas.

6.6 Validación de horarios solapados

Problema: El sistema debía impedir clases con horarios solapados, tanto para el mismo entrenador como para la agenda general.

Causa Raíz: La validación de clases no depende solo de los datos del formulario. También necesita consultar si el entrenador ya tiene otra clase asignada o si existe otra clase en el mismo horario.

Código relacionado:

```
private function validarReglasDeAgenda(Clase $clase, ?int $idIgnorado = null): void
{
    if (!$this->claseDAO->entrenadorExisteYDisponible($clase->getEntrenadorId())) {
        throw new DomainException('El entrenador asignado no existe o no esta disponible.');
```

```
    }

    if ($this->claseDAO->existeSolapamientoEntrenador(
        $clase->getDia(),
        $clase->getHora(),
        $clase->getDuracion(),
        $clase->getEntrenadorId(),
        $idIgnorado
    )) {
        throw new DomainException('El entrenador asignado no esta disponible en ese
horario.');
```

```
    }

    if ($this->claseDAO->existeSolapamientoGeneral(
        $clase->getDia(),
        $clase->getHora(),
        $clase->getDuracion(),
        $idIgnorado
    )) {
        throw new DomainException('El horario se solapa con otra clase existente.');
```

```
    }
}
```

Solución Implementada:

Se probaron los escenarios de solapamiento usando mocks de ClaseDAO. También se validó que, al editar una clase, se envíe su propio id como idIgnorado para evitar que la clase se compare consigo misma.

Código de prueba:

```
$claseDaoMock->expects($this->once())  
    ->method('existeSolapamientoEntrenador')  
    ->with($datos['dia'], $datos['hora'], $datos['duracion'], $datos['entrenadorId'], 7)  
    ->willReturn(false);  
  
$claseDaoMock->expects($this->once())  
    ->method('existeSolapamientoGeneral')  
    ->with($datos['dia'], $datos['hora'], $datos['duracion'], 7)  
    ->willReturn(false);
```

Resultado: Se comprobó que el sistema bloquea horarios conflictivos y permite editar una clase sin detectar un falso solapamiento consigo misma.

6.7 Control de cupos y reservas duplicadas

Problema: El sistema debía evitar reservas sin cupos disponibles, reservas duplicadas y reservas de atletas sin membresía vigente.

Causa Raíz: La reserva de una clase depende de varias condiciones: existencia del atleta, existencia de la clase, membresía pagada y vigente, cupos disponibles y ausencia de una reserva activa previa.

Código relacionado:

```
if (!$this->tieneMembresiaVigente($idAtleta)) {  
    throw new DomainException('El atleta no tiene una membresia pagada y vigente.');
```

```
}  
  
if ($this->claseDAO->consultarCupos($idClase) <= 0) {  
    throw new DomainException('La clase seleccionada no tiene cupos disponibles.');
```

```
}
```

```
if ($this->claseDAO->existeReservaActiva($idAtleta, $idClase)) {  
    throw new DomainException('El atleta ya tiene una reserva activa para esta clase.');
```

Solución Implementada: Se crearon pruebas en ClaseServiceReservasCuposTest.php para validar cada condición de forma independiente mediante mocks.

Resultado: Se comprobó que el sistema solo permite reservar cuando el atleta cumple todas las condiciones y que los cupos se liberan correctamente al cancelar una reserva.

7. CONCLUSIONES

7.1 Logros Alcanzados

100% de tests pasando (72/72).
100% de requisitos funcionales cubiertos (18/18).
9 suites de pruebas ejecutadas correctamente.
128 aserciones validadas.
Sistema de pruebas automatizado y reproducible mediante Composer y PHPUnit.
Ejecución rápida de la suite completa (< 2 segundos).
Validación del patrón Builder aplicado a Usuario, Membresía y Clase.
Uso de mocks para probar Services sin depender directamente de MySQL/Aiven.
Documentación completa del proceso de pruebas, resultados y problemas resueltos.

7.2 Calidad del Software

Confiabilidad: (5/5)

- 0 tests fallidos en la ejecución final.
- Resultados reproducibles mediante el comando composer test.
- Uso de datos de prueba controlados.
- Pruebas independientes de la conexión directa con MySQL/Aiven.
- Manejo correcto de excepciones esperadas en reglas de negocio.

Cobertura de Requisitos: (5/5)

- 18/18 requisitos funcionales cubiertos (100%).
- Módulo de usuarios validado: creación, edición, desactivación, autenticación, estado y rol.
- Módulo de membresías validado: creación, edición, asignación, pagos, consulta y vencimientos.
- Módulo de clases validado: creación, edición, eliminación, entrenadores, cupos y horarios.

- Casos de borde cubiertos: datos inválidos, duplicados, estados no permitidos, cupos agotados y horarios solapados.

Mantenibilidad: (5/5)

- Pruebas organizadas por Builders, Services y Controllers.
- Suites separadas por responsabilidad funcional.
- Uso de nombres descriptivos en los métodos de prueba.
- Fácil extensión para nuevos requisitos o módulos.
- Uso de inyección de dependencias para aislar la lógica de negocio.

Performance: (5/5)

- Suite completa ejecutada en aproximadamente 1.353 segundos.
- Pruebas unitarias rápidas al no depender de consultas reales a Aiven.
- Mocks de DAO reducen tiempo de ejecución.
- Sin pruebas lentas o dependientes de servicios externos.

7.3 Lecciones Aprendidas

1. PHPUnit se adapta correctamente a proyectos PHP nativos:

Permitió validar clases, servicios, excepciones y reglas de negocio sin requerir un framework adicional.

2. El patrón Builder facilita las pruebas unitarias:

Al concentrar validaciones de construcción en UsuarioBuilder, MembresiaBuilder y ClaseBuilder, fue posible probar datos válidos e inválidos de forma directa.

3. La inyección de dependencias facilita el uso de mocks:

Los Services reciben DAO por constructor, lo que permitió reemplazar UsuarioDAO, MembresiaDAO y ClaseDAO por objetos mock durante las pruebas.

4. No todas las pruebas deben depender de la base de datos:

Aunque el sistema usa MySQL en Aiven, las pruebas unitarias fueron más estables y seguras al simular la capa DAO.

5. Las pruebas de sesión requieren configuración especial:

Para evitar conflictos entre session_start() y la salida de PHPUnit, se utilizó la opción --stderr dentro del script composer test.

6. Las fechas en pruebas deben ser controladas:

En pruebas de clases se usaron fechas futuras dinámicas para evitar que los casos fallen con el paso del tiempo.

7. Las reglas de negocio cruzadas deben probarse por separado:

Casos como membresía vigente, cupos disponibles, reservas duplicadas y horarios solapados requieren pruebas específicas para cada condición.

7.4 Recomendaciones Futuras

1. Tests de Integración:

Agregar pruebas controladas contra una base de datos de pruebas en Aiven para validar el flujo completo Controller -> Service -> DAO -> MySQL.

2. Tests E2E:

Utilizar Playwright o Cypress para probar flujos completos desde la interfaz, como login, reserva de clase y registro de pago.

3. CI/CD:

Automatizar la ejecución de composer test en GitHub Actions para validar cada commit o pull request.

4. Cobertura de Código:

Incorporar reportes de cobertura con PHPUnit/Xdebug para medir qué porcentaje del código está cubierto por pruebas.

5. Pruebas de Seguridad:

Agregar pruebas específicas para sesiones, roles, acceso no autorizado y validación de entradas.

6. Pruebas de Rendimiento:

Medir tiempos de respuesta en operaciones críticas como listado de clases, reserva de cupos y generación de reportes.

7. Ampliación de Cobertura:

Extender el plan de pruebas a módulos adicionales del sistema, como progreso deportivo, comunicación interna y reportes administrativos.

Comandos útiles:

Instalar dependencias

composer install

Ejecutar todos los tests

composer test

Ejecutar PHPUnit directamente

vendor/bin/phpunit --configuration phpunit.xml --stderr --testdox tests

Ejecutar una suite específica

vendor/bin/phpunit --configuration phpunit.xml --stderr --testdox
tests/builders/UsuarioBuilderTest.php

7.5 Resumen Final

El plan de pruebas se completó exitosamente al 100%, cubriendo los 18 requisitos funcionales definidos en el tablero Kanban de IronClad Box mediante 72 pruebas automatizadas y 128 aserciones.

La suite permitió validar los módulos principales de usuarios, membresías y clases, así como el uso del patrón Builder para la construcción segura de entidades antes de llegar a la capa DAO. Además, el uso de mocks permitió comprobar la lógica de negocio sin depender directamente de la base de datos MySQL alojada en Aiven.

Puntos Destacados:

- Objetivo cumplido: 100% de tests pasando.
- Requisitos cubiertos: 18/18.
- Performance: suite completa ejecutada en aproximadamente 1.353 segundos.
- Robustez: pruebas reproducibles mediante composer test.
- Arquitectura validada: MVC por capas con patrón Builder.
- Documentación: informe completo con implementación, resultados, problemas y soluciones.

Estado Final: APROBADO - TODOS LOS REQUISITOS DEL KANBAN CUBIERTOS

ANEXOS

Anexo A: Comandos de Ejecución

```
# Instalar dependencias del proyecto
composer install

# Ejecutar todos los tests
composer test

# Ejecutar PHPUnit directamente
vendor/bin/phpunit --configuration phpunit.xml --stderr --testdox tests

# Ejecutar una suite específica de Builders
vendor/bin/phpunit --configuration phpunit.xml --stderr --testdox
tests/builders/UsuarioBuilderTest.php

# Ejecutar una suite específica de Services
vendor/bin/phpunit --configuration phpunit.xml --stderr --testdox
tests/services/ClaseServiceReservasCuposTest.php

# Ver versión de PHP
php -v

# Ver versión de PHPUnit
vendor/bin/phpunit --version
```

Anexo B: Estructura de Test Típica

```
use PHPUnit\Framework\TestCase;
```

```

final class NombreDelComponenteTest extends TestCase
{
    public function test_debe_realizar_una_accion_esperada(): void
    {
        // Arrange: preparar datos, mocks o entidades necesarias
        $daoMock = $this->createMock(UsuarioDAO::class);

        $daoMock->method('buscarPorCorreo')
            ->willReturn($usuario);

        $service = new UsuarioService($daoMock);

        // Act: ejecutar el método que se desea probar
        $resultado = $service->autenticar('usuario@example.com', 'claveSegura1');

        // Assert: verificar el resultado esperado
        $this->assertSame('Atleta', $resultado->getRol());
    }
}

```

Anexo C: Configuración Completa

```

composer.json:
{
    "name": "ironbox/v1",
    "description": "Proyecto IronBox",
    "type": "project",
    "require": {
        "php": ">=8.1"
    },
    "require-dev": {

```

```

    "phpunit/phpunit": "^9.6"
  },
  "autoload": {
    "classmap": [
      "builders/",
      "controllers/",
      "dao/",
      "includes/",
      "models/",
      "services/"
    ]
  },
  "autoload-dev": {
    "classmap": [
      "tests/"
    ]
  },
  "scripts": {
    "test": "phpunit --configuration phpunit.xml --stderr --testdox tests"
  }
}

```

phpunit.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap="tests/bootstrap.php" colors="true">
  <testsuites>
    <testsuite name="IronClad Box Test Suite">
      <directory>tests</directory>
    </testsuite>
  </testsuites>

```

</phpunit>

Anexo D: Métricas Detalladas

Por Categoría:

- Builders: 29 tests (40.3%)
- Services: 38 tests (52.8%)
- Controllers: 5 tests (6.9%)

Por Archivo:

- UsuarioBuilderTest.php: 9 tests
- MembresiaBuilderTest.php: 10 tests
- ClaseBuilderTest.php: 10 tests
- UsuarioServiceAutenticacionTest.php: 6 tests
- UsuarioServiceEdicionTest.php: 7 tests
- MembresiaServiceEstadoTest.php: 5 tests
- MembresiaServiceAsignacionTest.php: 6 tests
- ClaseServiceAgendaTest.php: 6 tests
- ClaseServiceReservasCuposTest.php: 8 tests
- AuthControllerTest.php: 5 tests

Por Módulo Funcional:

- Usuarios: 27 tests
- Membresías: 25 tests
- Clases: 20 tests

Por Tipo de Validación:

- Creación y construcción de entidades: 29 tests
- Validación de datos inválidos: 22 tests
- Manejo de excepciones de negocio: 18 tests
- Autenticación y sesión: 5 tests
- Control de cupos y reservas: 8 tests
- Validación de horarios y agenda: 6 tests
- Uso de mocks e inyección de dependencias: aplicado en Services y AuthControllerTest

Resultado Final:

- Tests ejecutados: 72/72
- Aserciones: 128
- Suites: 9/9
- Tasa de éxito: 100%

Anexo E: Evidencia de Ejecución

Resultado de la ejecución del comando:

composer test

PHPUnit 9.6.35 by Sebastian Bergmann and contributors.

Clase Builder

- ✓ Crea clase valida
- ✓ Rechaza id invalido
- ✓ Rechaza fecha invalida
- ✓ Rechaza hora invalida
- ✓ Rechaza fecha u hora pasada
- ✓ Rechaza duracion cero o negativa
- ✓ Rechaza duracion mayor a 240
- ✓ Rechaza cupo mayor a capacidad
- ✓ Rechaza entrenador invalido
- ✓ Rechaza cupos disponibles mayores al cupo maximo

Membresia Builder

- ✓ Crea membresia valida
- ✓ Rechaza atleta invalido
- ✓ Rechaza tipo vacio
- ✓ Rechaza precio cero o negativo
- ✓ Redondea precio a dos decimales
- ✓ Rechaza fecha inicio invalida
- ✓ Calcula vencimiento a treinta dias
- ✓ Marcar como pagado actualiza estado inicio y vencimiento
- ✓ Rechaza estado invalido
- ✓ Rechaza vencimiento anterior a inicio

Usuario Builder

- ✓ Crea usuario con datos validos
- ✓ Rechaza nombre menor a tres caracteres
- ✓ Rechaza cedula ecuatoriana invalida
- ✓ Rechaza correo invalido
- ✓ Hashea contrasena correctamente
- ✓ Rechaza contrasena menor a ocho caracteres
- ✓ Rechaza rol invalido
- ✓ Rechaza estado invalido
- ✓ Construir falla si faltan datos obligatorios

Auth Controller

- ✓ Login administrador devuelve rol administrador
- ✓ Login entrenador devuelve rol entrenador
- ✓ Login atleta devuelve rol atleta
- ✓ Me sin sesion devuelve no autenticado
- ✓ Logout cierra sesion

Clase Service Agenda

- ✓ Eliminar clase existente
- ✓ Eliminar clase inexistente lanza error
- ✓ Crear clase falla si entrenador no existe o no disponible
- ✓ Crear clase falla si hay solapamiento del entrenador
- ✓ Crear clase falla si hay solapamiento general
- ✓ Editar clase ignora su propio id al validar solapamiento

Clase Service Reservas Cupos

- ✓ Listar clases disponibles valida atleta
- ✓ Reservar cupo con membresia pagada y vigente
- ✓ Reservar falla si atleta no existe
- ✓ Reservar falla si clase no existe

- ✓ Reservar falla si membresia no esta vigente
- ✓ Reservar falla si no hay cupos
- ✓ Reservar falla si ya existe reserva activa
- ✓ Cancelar reserva libera cupo

Membresia Service Asignacion

- ✓ Crea membresia para atleta existente
- ✓ Crear falla si atleta no existe
- ✓ Crear calcula vencimiento si no se envia
- ✓ Actualizar membresia existente
- ✓ Actualizar falla si membresia no existe
- ✓ Obtener actual por atleta rechaza id invalido

Membresia Service Estado

- ✓ Registrar pago por id membresia cambia a pagado
- ✓ Registrar pago por id atleta busca membresia actual
- ✓ Registrar pago falla si no existe membresia
- ✓ Cancelar membresia existente
- ✓ Solicitar membresia falla si ya tiene pendiente o pagada

Usuario Service Autenticacion

- ✓ Autentica usuario activo con credenciales validas
- ✓ Rechaza correo vacio o contrasena vacia
- ✓ Rechaza contrasena incorrecta
- ✓ Rechaza usuario inexistente
- ✓ Rechaza usuario inactivo
- ✓ Normaliza correo antes de autenticar

Usuario Service Edicion

- ✓ Edita usuario con datos validos
- ✓ Editar falla si usuario no existe

- ✓ Editar mantiene contraseña si no se envía nueva
- ✓ Editar hash de contraseña si se envía nueva
- ✓ Editar rechaza correo usado por otro usuario
- ✓ Editar rechaza cédula usada por otro usuario
- ✓ Desactivar usuario activo

Time: 00:01.544, Memory: 8.00 MB

OK (72 tests, 128 assertions)

FIN DEL INFORME

Documento generado en julio de 2026.

Sistema IronClad Box v1.0 - Plan de Pruebas Unitarias Completo.

Todos los requisitos funcionales definidos en el tablero Kanban (REQ-001 a REQ-018) fueron implementados y validados mediante PHPUnit.